

Link da atividade no ava: [Atividades Autodirigidas](#)

Desafio:

A NogTech, plataforma de cursos de tecnologia, precisa consolidar um relatório diário cruzando **transações financeiras** com **engajamento dos alunos**. A diretoria quer saber, em uma única visão:

Quem está pagando pelos cursos, de onde vem (geograficamente), em quais datas (incluindo feriados) e se essas pessoas estão realmente consumindo a plataforma.

O ecossistema deve ser **tolerante a falhas**, garantir **independência** (reexecuções sem duplicar dados) e aplicar **anonimização em conformidade com a LGPD**.

Escopo

O pipeline deve consolidar um relatório de vendas diárias integrando dados locais e governamentais.

Extract (Extração)

O pipeline deve extrair dados de duas fontes distintas:

1. **Fonte A (Arquivos Estáticos):** Arquivos locais chamados:
 - transacoes_nogtech.csv — exportado pelo ERP financeiro (encoding latin-1, delimitador ;)
 - engajamento_alunos.json — exportado pela plataforma de vídeo (encoding utf-8)
2. **Fonte B (API em Tempo Real):** Consumir dados públicos diretamente da **BrasilAPI** (sem necessidade de autenticação/tokens) para enriquecer o contexto do pipeline.

Transform (Transformação)

Os dados extraídos devem ser cruzados, validados e higienizados de acordo com as seguintes regras:

- Os dados devem ser cruzados, validados e higienizados conforme as regras abaixo. **Todas as regras são obrigatórias.**

3.1 Junção das fontes locais Cruzar transações com engajamento usando `cpf_aluno` como chave. Para cada transação, anexar o engajamento do **mês correspondente** (`mes_referencia` = mês de `data_transacao`). Quando não houver engajamento para o mês, manter a transação com campos de engajamento nulos (LEFT JOIN).

3.2 Padronização de CPF CPFs aparecem em dois formatos no arquivo (123.456.789-00 e 12345678900). Padronize todos para o formato **com máscara** antes de aplicar a anonimização.

3.3 Enriquecimento de Localização (BrasilAPI) Usar <https://brasilapi.com.br/api/cep/v2/{CEP}> para converter `cep_cobranca` em **Cidade, Estado (UF) e Bairro**, adicionando esses três campos ao registro. **Implementar cache local** (dicionário em memória, arquivo JSON ou SQLite) — não consultar a API duas vezes para o mesmo CEP.

3.4 Análise de Calendário (BrasilAPI) Usar <https://brasilapi.com.br/api/feriados/v1/{ANO}> para verificar se `data_transacao` ocorreu em feriado nacional. Criar a coluna booleana `venda_em_feriado` (true/false). **Cachear a lista de feriados por ano** — uma única chamada por ano cobre todas as transações daquele ano.

3.5 Anonimização (LGPD) Antes de gravar no destino, o campo `cpf_aluno` deve ser **mascarado** mantendo apenas os 6 dígitos centrais visíveis:

- Entrada: 123.456.789-00
- Saída: ***.456.789-**

- O campo nome_aluno deve ser **removido** do dataset final (não basta mascarar — nomes são identificadores diretos).

3.6 Idempotência O pipeline deve ser projetado para ser executado múltiplas vezes para o mesmo lote sem gerar duplicidade. Estratégias aceitas (escolher uma e justificar no README):

- Chave natural (id_transacao) + UPSERT no banco
- Particionamento por data_transacao com overwrite da partição
- Hash da linha + tabela de controle de lotes processados

Load (Carga)

O dado final transformado e enriquecido deve ser gravado em **uma** das opções:

- **Banco relacional** (PostgreSQL ou MySQL) com tabela fato_vendas modelada adequadamente
- **Arquivo estruturado** em **Parquet** particionado por ano/mês (simulando uma camada de Data Lake)

Requisitos de Engenharia de Software e Infraestrutura

- **Containerização (Docker):** A ferramenta de orquestração escolhida (Airflow, NiFi ou Luigi), bem como quaisquer bancos de dados auxiliares utilizados, devem ser inicializados e orquestrados exclusivamente via docker-compose.yml.
- **Idempotência:** O pipeline deve ser projetado para ser executado múltiplas vezes para o mesmo lote de dados sem gerar duplicidade ou inconsistência no destino final (carga incremental ou sobreescrita limpa).
- **Tratamento de Erros e Resiliência:** Caso a BrasilAPI fique temporariamente instável ou ocorra um erro de rede, o pipeline não pode corromper os dados. Deve ser implementada uma

política simples de tentativas (*retry*) ou um desvio de fluxo para registrar falhas críticas em logs estruturados.

- **Observabilidade:** O painel web do orquestrador escolhido deve exibir claramente o grafo de execução, o histórico das execuções passadas e os tempos de execução de cada nó.
- OBS:A BrasilAPI é pública e gratuita. **Respeite-a:** use cache, agrupe requisições e não rode o pipeline em loop infinito durante o desenvolvimento.

Opções de Implementação (A equipe deve escolher apenas uma)

Opção A: Apache Airflow (Código como Configuração)

- Implementação das tarefas utilizando operadores adequados (ex: `PythonOperator`, `BashOperator` ou `SqlOperator`).
- Uso correto de dependências no ecossistema Python (ex: `task_extract >> task_transform >> task_load`).
- Gerenciamento correto de pequenos metadados entre tarefas, se necessário, usando `XComs`.

Opção B: Apache NiFi (Fluxo Baseado em Eventos)

- Uso correto dos Processors nativos para chamadas HTTP e manipulação de arquivos (ex: `InvokeHTTP`, `EvaluateJsonPath`, `UpdateAttribute`, `PutSQL`).
- Organização visual utilizando *Process Groups* para garantir a modularidade e legibilidade do Canvas.
- Configuração adequada das filas de conexão e políticas de *Backpressure*.

Opção C: Luigi (Orientado a Alvos/Outputs)

- Modelagem das etapas herdando diretamente da classe `luigi.Task`.
- Uso explícito do método `requires()` para estruturar a árvore de dependências de trás para frente.
- Utilização correta de `luigi.Target` no método `output()` para garantir que o framework identifique se a tarefa foi previamente concluída, evitando reprocessamentos desnecessários.
- Execução conectada e monitorada pelo **Luigi Central Scheduler** (porta 8082).

Entregáveis

1. **Repositório Git (GitHub/GitLab):** Contendo todo o código-fonte, DAGs, configurações do NiFi ou scripts Luigi, além do arquivo `docker-compose.yml` unificado.
2. **README.md (Documentação Técnica):**
 - Instruções de inicialização do ambiente (`docker-compose up`).
 - Portas de acesso às interfaces visuais.
 - Explicação sucinta da estratégia utilizada para garantir a idempotência e o tratamento de falhas.
3. **Demonstração em Vídeo (Máx. 5 minutos):** Um vídeo curto mostrando o ambiente subindo, o pipeline sendo disparado com sucesso através da interface visual da ferramenta escolhida e o resultado final persistido no destino.

